Adendo Full-Stack: Voz, CORS e Resolução

https://gemini.google.com/app/9543196526c84ac3

*User prompt: Atue como um Engenheiro de Software Sênior e Professor Especialista em Desenvolvimento Web Full-Stack. Preciso que você crie um "Adendo de Extensibilidade e Resolução de Erros" para incorporar ao Guia de Sobrevivência Full-Stack da minha prova prática. O conteúdo deve ser tecnicamente preciso, sem reticências, totalmente funcional e ricamente comentado linha por linha. Adicione estritamente as seguintes seções ao material: --- ### ADENDO 1: RECURSO DE COMANDOS DE VOZ NO FRONTEND (Web Speech API) - Forneça um componente React limpo ou uma função utilitária utilizando a API nativa do navegador (`window. SpeechRecognition` ou `window.webkitSpeechRecognition`). - O código deve capturar a voz do usuário em português do Brasil ('pt-BR'), converter em texto e preencher automaticamente o input de criação de tarefas, disparando a adição ou apenas atualizando o estado do input para validação do usuário. Inclua tratamento de erro caso o navegador não suporte a API. --- ### ADENDO 2: REFORÇO DE CONFIGURAÇÃO DE CORS E SOCKET.IO - Explique tecnicamente e traga o bloco exato de código mostrando a dependência mútua entre o CORS do Express (`app. use(cors({ origin, credentials: true }))`) e a inicialização do Socket.io (`new Server(httpServer, { cors: { origin, credentials: true } })`). - Destaque o motivo pelo qual a ausência do objeto de credenciais em ambos quebra o WebSocket silenciosamente. --- ### ADENDO 3: INTERCEPTOR DE RESPOSTA DO AXIOS (Tratamento Global de Sessão Expirada) - Atualize o arquivo `src/services/api. js` adicionando um *response interceptor* do Axios. - Explique como interceptar erros de resposta e verificar se o status HTTP é `401 Unauthorized` (indicando que o token JWT expirou ou o cookie sumiu). Caso ocorra, o interceptor deve redirecionar automaticamente o usuário para a tela de login (`window.location.href = '/login'`), prevenindo que a interface da aplicação trave em um estado quebrado. --- ### ADENDO 4: GUIA DE DECISÃO E RESOLUÇÃO DE PROBLEMAS (FAQ DE PROVA) Crie um guia prático em formato de perguntas e respostas para o momento de "branco" na prova:1. **"O que fazer se tal coisa deu erro? (Checklist de Emergência):"** - Roteiro passo a passo do que checar quando a API não comunica com o front (porta do Vite vs porta do Express, presença do `withCredentials: true`, prefixo `/api` faltando nas rotas do Axios). - O que fazer se o banco de dados não responde (verificação do comando do MongoDB, checagem do arquivo `.env`). 2. **"Quero adicionar uma nova funcionalidade (ex:categorias, comentários ou favoritos). Qual o fluxo exato de desenvolvimento entre as camadas?"** - Passo a passo da ordem correta para criar do zero: Model (Schema Mongoose) -> Controller (Regras de negócio e status HTTP) -> Rotas (Routes acoplando o middleware de proteção) -> Consumo no Frontend (Axios Service + Estado React + Interface com Tailwind v4).*

Response: Aqui está o seu **Adendo de Extensibilidade e Resolução de Erros**, elaborado com precisão técnica e rigor arquitetural para ser incorporado diretamente ao seu *Guia de Sobrevivência Full-Stack*.

Todo o código é totalmente funcional, sem omissões ou reticências, e está comentado linha por linha para servir como material de consulta e justificativa técnica durante a sua avaliação.

ADENDO DE EXTENSIBILIDADE E RESOLUÇÃO DE ERROS

Suplemento Oficial para o Guia de Sobrevivência Full-Stack (Node.js + React)

ADENDO 1: RECURSO DE COMANDOS DE VOZ NO FRONTEND (Web Speech API)

A **Web Speech API** permite a transcrição de áudio em texto em tempo real diretamente no navegador, sem a necessidade de dependências externas ou chaves de API pagas. Utiliza-se a interface `SpeechRecognition` (ou o prefixo `webkitSpeechRecognition` em navegadores baseados em Chromium).

Abaixo está a implementação do componente React `ComandoVozInput.jsx`, projetado para capturar a fala do usuário em português do Brasil (`pt-BR`), atualizar o estado do input de criação de tarefas e oferecer a opção de disparo automático ou revisão manual.

Arquivo: `frontend/src/components/ComandoVozInput.jsx`

```
// Importa os hooks do React necessários para gerenciamento de estado e ciclo de vida
import { useState, useEffect, useRef } from 'react';

// Componente utilitário de entrada de texto com suporte nativo a comandos de voz
export default function ComandoVozInput({ valorInput, setValorInput, aoSubmeter }) {
  // Estado que indica se o navegador está ouvindo a voz do usuário no momento
  const [ouvindo, setOuvindo] = useState(false);

  // Estado para armazenar mensagens de erro caso a API não seja suportada ou haja falha na captura
  const [erroVoz, setErroVoz] = useState('');

  // Referência para armazenar a instância do reconhecedor de voz sem provocar re-renderizações
  const mefReconhecimento = useRef(null);

  // Hook de efeito executado uma única vez na montagem para inicializar a API de voz
  useEffect(() => {
    // Captura a implementação nativa ou a versão prefixada do WebKit
    const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;

    // Valida se o navegador atual possui suporte para a Web Speech API
    if (!SpeechRecognition) {
      setErroVoz('Reconhecimento de voz não é suportado neste navegador.');
      return;
    }

    // Instancia o objeto de reconhecimento de fala
    const reconhecimento = new SpeechRecognition();

    // Configura a captura contínua como falsa (para após detectar a pausa da fala)
    reconhecimento.continuous = false;

    // Desativa resultados parciais intermediários, retornando apenas a transcrição final
```

```
  reconhecimento.interimResults = false;

  // Define estritamente o idioma de captura para o Português do Brasil
  reconhecimento.lang = 'pt-BR';

  // Evento disparado quando a API reconhece com sucesso um trecho falado
  reconhecimento.onresult = (evento) => {
    // Extrai o texto transcrito do primeiro resultado retornado
    const textoTranscrito = evento.results[0][0].transcript;

    // Atualiza o estado do input pai com o texto convertido da voz
    setValorInput(textoTranscrito);

    // Reseta eventuais mensagens de erro anteriores
    setErroVoz('');
  };

  // Evento disparado caso ocorra algum erro no processo de captura de áudio
  reconhecimento.onerror = (evento) => {
    // Trata os tipos comuns de erro da Web Speech API
    if (evento.error === 'no-speech') {
      setErroVoz('Nenhuma fala foi detectada. Tente novamente.');
    } else if (evento.error === 'audio-capture') {
      setErroVoz('Microfone não encontrado ou desativado.');
    } else if (evento.error === 'not-allowed') {
      setErroVoz('Permissão de acesso ao microfone foi negada.');
    } else {
      setErroVoz('Erro no reconhecimento de voz: ' + evento.error);
    }
    // Atualiza o estado de escuta para falso
    setOuvindo(false);
  };

  // Evento disparado quando o serviço de voz encerra a captura
  reconhecimento.onend = () => {
    // Garante que o indicador visual de escuta seja desativado
    setOuvindo(false);
  };

  // Guarda a instância configurada na referência persistente
  mefReconhecimento.current = reconhecimento;
}, [setValorInput]);

// Função disparada ao clicar no botão de alternar escuta
const alternarEscuta = () => {
  // Se a API não estiver disponível no navegador, interrompe a execução
  if (!mefReconhecimento.current) {
    setErroVoz('Recurso indisponível neste navegador.');
    return;
  }

  // Se já estiver ouvindo, encerra a captura manualmente
  if (ouvindo) {
    mefReconhecimento.current.stop();
    setOuvindo(false);
  } else {
    // Limpa erros anteriores e inicia a escuta do microfone
    setErroVoz('');
    try {
      mefReconhecimento.current.start();
      setOuvindo(true);
    } catch (err) {
      // Trata a exceção caso a captura já estivesse em processo de inicialização
      setErroVoz('Falha ao iniciar microfone. Tente novamente.');
    }
  }
};

return (
  <div className="w-full space-y-2">
    {/* Container flexível combinando a caixa de texto e os botões de ação */}
    <div className="flex gap-2">
      {/* Input de texto controlado pelo componente pai (Dashboard) */}
      <input
        type="text"
        value={valorInput}
        onChange={(e) => setValorInput(e.target.value)}
        placeholder="Digite ou clique no microfone para falar..."
        className="flex-1 rounded-lg border border-gray-300 px-3 py-2 focus:border-blue-500 focus:ring-2 focus:rin
```

```
        />

        {/* Botão de ativação do microfone por voz */}
        <button
          type="button"
          onClick={alternarEscuta}
          className={`flex items-center gap-1 rounded-lg px-3 py-2 text-sm font-semibold text-white transition ${
            ouvindo ? 'animate-pulse bg-red-600 hover:bg-red-700' : 'bg-purple-600 hover:bg-purple-700'
          }`}
          title="Capturar comando de voz"
        >
          {/* Ícone textual indicando o estado atual da gravação */}
          {ouvindo ? '🎙️ Ouvindo...' : '🎤 Voz'}
        </button>

        {/* Botão padrão de submissão do formulário de criação de tarefa */}
        <button
          type="button"
          onClick={aoSubmeter}
          className="rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white transition hover:bg-blue-700"
        >
          Adicionar
        </button>
      </div>

      {/* Exibição condicional de mensagem de erro da API de voz */}
      {erroVoz && (
        <p className="rounded-md border border-amber-300 bg-amber-50 px-3 py-1 text-xs text-amber-800">
          {erroVoz}
        </p>
      )}
    </div>
  );
}
```

ADENDO 2: REFORÇO DE CONFIGURAÇÃO DE CORS E SOCKET.IO

Análise Técnica da Dependência Mútua de Credenciais

Quando a aplicação utiliza autenticação baseada em **cookies** `httpOnly` ou headers autenticados entre origens cruzadas (ex: frontend em `http://localhost:5173` e backend em `http://localhost:3000`), tanto o Express quanto a instância do Socket.io realizam verificações estritas do protocolo CORS (*Cross-Origin Resource Sharing*).

1. **Separação de Camadas**: O Express trata as requisições HTTP REST convencionais. O Socket.io lida com o *handshake* inicial HTTP Upgrade, convertendo a conexão para o protocolo WebSocket (`ws://` ou `wss://`).
2. **O Papel do `credentials: true`**: Quando `credentials: true` é configurado no backend, o servidor responde com o cabeçalho HTTP `Access-Control-Allow-Credentials: true`. Isso autoriza o navegador a enviar cookies e headers de autorização em requisições *cross-origin*.
3. **Mecanismo da Falha Silenciosa no WebSocket**:
   - Ao inicializar a conexão WebSocket, o cliente do Socket.io realiza uma requisição HTTP de Polling inicial enviando os cookies de sessão armazenados no navegador (`withCredentials: true`).
   - Se o motor interno do servidor Socket.io **não** possuir a configuração explícita de `cors: { origin, credentials: true }`, ele rejeitará a validação do *handshake* no nível HTTP.
   - O servidor **não lança uma exceção no Node.js**; em vez disso, retorna um status HTTP `403 Forbidden` ou recusa os cabeçalhos de *upgrade*.
   - No cliente, o Socket.io entra em um ciclo infinito e silencioso de tentativas de reconexão (`reconnect_attempt`), fazendo com que o recurso em tempo real pare de funcionar sem emitir erros claros de sintaxe no servidor.

Bloco Exato de Configuração (`backend/src/server.js`)

```
// Importa as dependências necessárias para o servidor web e o WebSocket
import express from 'express';
import http from 'http';
import cors from 'cors';
import { Server } from 'socket.io';

// Inicializa a aplicação Express
const app = express();

// Cria o servidor HTTP nativo do Node.js encapsulando o servidor Express
const httpServer = http.createServer(app);

// Define a origem exata do cliente React (NUNCA utilize '*' quando credentials for true)
const CLIENT_URL = process.env.CLIENT_URL || 'http://localhost:5173';
```

```
// ---------------------------------------------------------------------------
// 1. CONFIGURAÇÃO DE CORS NO EXPRESS (Para Rotas REST /api)
// ---------------------------------------------------------------------------
app.use(cors({
  origin: CLIENT_URL,      // Autoriza estritamente a URL do frontend Vite
  credentials: true        // Permite o recebimento e envio de cookies httpOnly
}));

app.use(express.json());

// ---------------------------------------------------------------------------
// 2. CONFIGURAÇÃO DE CORS NO SOCKET.IO (Para Handshake e WebSockets)
// ---------------------------------------------------------------------------
// O Socket.io encapsula seu próprio manipulador HTTP/CORS. Ele NÃO herda
// automaticamente as configurações declaradas no app.use(cors()) do Express.
const io = new Server(httpServer, {
  cors: {
    origin: CLIENT_URL,    // Deve corresponder exatamente à mesma URL do Express
    credentials: true      // Permite a validação de cookies no Handshake inicial
  }
});

// Guarda a instância do Socket.io na aplicação para uso nos Controllers
app.set('io', io);

// Inicializa o servidor escutando a porta configurada
httpServer.listen(3000, () => {
  console.log('Servidor e Socket.io executando em http://localhost:3000');
});
```

ADENDO 3: INTERCEPTOR DE RESPOSTA DO AXIOS (Tratamento Global de Sessão Expirada)

Em aplicações *Single Page Applications* (SPA), quando a sessão do usuário expira (token JWT vencido ou remoção do cookie `httpOnly`), requisições subsequentes à API retornam o código de status HTTP `401 Unauthorized`.

Sem um tratamento centralizado, cada componente individual precisaria tratar o erro `401`, resultando em estados de interface congelados, botões travados em carregamento ou falhas silenciosas. O **Response Interceptor** do Axios captura qualquer resposta da API antes que ela chegue ao bloco `catch` do componente chamador.

Arquivo Atualizado: `frontend/src/services/api.js`

```
// Importa o cliente HTTP Axios
import axios from 'axios';

// Cria a instância base configurada da API
const api = axios.create({
  // Define a URL base da API REST obtida das variáveis de ambiente do Vite
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000/api',

  // Habilita o envio automático dos cookies httpOnly nas requisições cross-origin
  withCredentials: true,

  // Define o tipo de conteúdo padrão como JSON
  headers: { 'Content-Type': 'application/json' },

  // Tempo limite para abortar a requisição caso o servidor não responda
  timeout: 10000,
});

// ---------------------------------------------------------------------------
// INTERCEPTOR DE RESPOSTA DO AXIOS (Global Response Interceptor)
// ---------------------------------------------------------------------------
api.interceptors.response.use(
  // Primeiro parâmetro: Executado para respostas bem-sucedidas (Status HTTP 2xx)
  (resposta) => {
    // Retorna a resposta sem alterações para ser consumida pelo .then() ou await
    return resposta;
  },

  // Segundo parâmetro: Executado para respostas de erro (Status HTTP fora da faixa 2xx)
  (erro) => {
    // Captura o código de status HTTP retornado pelo servidor, se houver resposta
    const statusHTTP = erro.response ? erro.response.status : null;

    // Identifica se a falha é um erro 401 (Não Autorizado / Sessão Expirada)
    if (statusHTTP === 401) {
      // Obtém o caminho atual da rota do navegador para evitar loops infinitos
      const caminhoAtual = window.location.pathname;
```

```
      // Se o usuário não estiver na tela de login ou de cadastro, força o redirecionamento
      if (caminhoAtual !== '/login' && caminhoAtual !== '/cadastro') {
        console.warn('Sessão expirada ou inválida. Redirecionando para a tela de login...');

        // Redireciona a aplicação inteira para a rota de login via navegador
        window.location.href = '/login';
      }
    }

    // Rejeita a promessa mantendo o fluxo de erro disponível para tratamento local se necessário
    return Promise.reject(erro);
  }
);

// Função auxiliar utilitária para extrair mensagens de erro enviadas pelo backend
export function mensagemDeErro(erro) {
  return erro.response?.data?.message || 'Não foi possível conectar ao servidor';
}

// Exporta a instância do Axios como exportação padrão
export default api;
```

ADENDO 4: GUIA DE DECISÃO E RESOLUÇÃO DE PROBLEMAS (FAQ DE PROVA)

1. Checklist de Emergência: "O que fazer se der erro?"

Cenario A: Frontend não comunica com a API (Erro de Rede, CORS ou 404)

1. **Verificação de Portas**:
   - Confirme se a API está rodando na porta `3000` e o Vite na porta `5173`.
   - No `frontend/.env`, garanta que `VITE_API_URL=http://localhost:3000/api`.
   - Se alterou o `.env` do Vite, **pare o processo do terminal (`Ctrl + C`) e rode `npm run dev` novamente** (o Vite não recarrega variáveis de ambiente em tempo de execução).
2. **Presença do `withCredentials: true` e `credentials: true`**:
   - Se o console exibir erro de CORS, certifique-se de que a `CLIENT_URL` no `.env` do backend é exatamente `http://localhost:5173` (sem barra no final).
   - Confirme que o Axios possui `withCredentials: true` no `api.js` e o Express possui `app.use(cors({ origin: CLIENT_URL, credentials: true }))`.
3. **Prefixos de Rota Soltos (`/api`)**:
   - Se a requisição retornar `404 Not Found` no console, cheque se a chamada Axios não esqueceu a barra inicial ou o prefixo: `api.get('/tasks')` chama `http://localhost:3000/api/tasks`.
   - No `server.js`, verifique se o grupo de rotas foi montado sob o prefixo correto: `app.use('/api/tasks', taskRoutes)`.

Cenario B: O Banco de Dados (MongoDB) não responde

1. **Verificação do Serviço**:
   - Execute no terminal o comando de teste rápido: `mongosh --eval 'db.runCommand({ ping: 1 })'`
   - Se o comando falhar, inicie o MongoDB manualmente:
     - **Windows (PowerShell Admin)**: `net start MongoDB`
     - **Linux**: `sudo systemctl start mongod`
     - **macOS**: `brew services start mongodb-community`
2. **Arquivo `.env` do Backend**:
   - Abra o `backend/.env` e verifique a string de conexão: `MONGO_URI=mongodb://127.0.0.1:27017/prova_db`.
   - Prefira utilizar a sintaxe de IP numérico `127.0.0.1` em vez da palavra `localhost` para evitar problemas de resolução DNS IPv6 (`::1`) no Node.js 18+.

2. Fluxo Exato de Desenvolvimento de Novas Funcionalidades

Quando a prova solicitar uma nova funcionalidade completa (exemplo: **Favoritos** ou **Categorias**), siga rigorosamente a ordem arquitetural em 4 etapas:

```
[1. MODEL] ──> [2. CONTROLLER] ──> [3. ROUTES] ──> [4. FRONTEND]
  Mongoose          Regras &          Express        Axios, Estado
   Schema          Status HTTP       Middlewares    & Tailwind CSS
```

Passo 1: Model (`backend/src/models/Favorito.js`)

Crie o esquema de dados no Mongoose com as validações necessárias e referências aos IDs existentes.

```
import mongoose from 'mongoose';

const favoritoSchema = new mongoose.Schema(
  {
    titulo: { type: String, required: [true, 'Título obrigatório'], trim: true },
    url: { type: String, required: [true, 'URL obrigatória'] },
    owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, immutable: true }
  },
  { timestamps: true }
);

export default mongoose.model('Favorito', favoritoSchema);
```

Passo 2: Controller (`backend/src/controllers/favoritoController.js`)

Implemente as regras de negócio, extraia o dono do token (`req.userId`) e retorne os status HTTP adequados (`201` para criação, `200` para leitura, `400` para validação).

```
import Favorito from '../models/Favorito.js';

export async function criarFavorito(req, res) {
  try {
    const { titulo, url } = req.body;
    if (!titulo || !url) {
      return res.status(400).json({ message: 'Título e URL são obrigatórios' });
    }
    const favorito = await Favorito.create({ titulo, url, owner: req.userId });
    return res.status(201).json(favorito);
  } catch (erro) {
    return res.status(500).json({ message: 'Erro ao salvar favorito' });
  }
}

export async function listarFavoritos(req, res) {
  try {
    const favoritos = await Favorito.find({ owner: req.userId }).sort({ createdAt: -1 });
    return res.status(200).json(favoritos);
  } catch (erro) {
    return res.status(500).json({ message: 'Erro ao buscar favoritos' });
  }
}
```

Passo 3: Routes (`backend/src/routes/favoritoRoutes.js`) e Registro no `server.js`

Crie o roteador Express, aplique o middleware de proteção JWT e registre o prefixo da rota no servidor principal.

```
// backend/src/routes/favoritoRoutes.js
import { Router } from 'express';
import { criarFavorito, listarFavoritos } from '../controllers/favoritoController.js';
import { protegerRota } from '../middlewares/authMiddleware.js';

const router = Router();
router.use(protegerRota);

router.post('/', criarFavorito);
router.get('/', listarFavoritos);

export default router;
```

```
// Em backend/src/server.js:
import favoritoRoutes from './routes/favoritoRoutes.js';
app.use('/api/favoritos', favoritoRoutes);
```

Passo 4: Consumo no Frontend (`frontend/src/components/ListaFavoritos.jsx`)

Consuma os endpoints através do serviço do Axios (`api.js`), gerencie o estado local do React (`useState`, `useEffect`) e construa a interface responsiva utilizando classes utilitárias do Tailwind CSS.

```
import { useEffect, useState } from 'react';
import api, { mensagemDeErro } from '../services/api.js';

export default function ListaFavoritos() {
  const [favoritos, setFavoritos] = useState([]);
  const [form, setForm] = useState({ titulo: '', url: '' });
  const [erro, setErro] = useState('');
```

```
  const carregarFavoritos = async () => {
    try {
      const res = await api.get('/favoritos');
      setFavoritos(res.data);
    } catch (err) {
      setErro(mensagemDeErro(err));
    }
  };

  useEffect(() => {
    carregarFavoritos();
  }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const res = await api.post('/favoritos', form);
      setFavoritos((prev) => [res.data, ...prev]);
      setForm({ titulo: '', url: '' });
    } catch (err) {
      setErro(mensagemDeErro(err));
    }
  };

  return (
    <div className="space-y-4 rounded-xl bg-white p-6 shadow-md">
      <h2 className="text-lg font-bold text-gray-800">Meus Favoritos</h2>
      {erro && <p className="rounded-lg border border-red-300 bg-red-50 p-2 text-sm text-red-700">{erro}</p>}

      <form onSubmit={handleSubmit} className="flex gap-2">
        <input
          type="text"
          placeholder="Título"
          value={form.titulo}
          onChange={(e) => setForm({ ...form, titulo: e.target.value })}
          className="rounded-lg border border-gray-300 px-3 py-2 text-sm focus:outline-hidden"
          required
        />
        <input
          type="url"
          placeholder="https://exemplo.com"
          value={form.url}
          onChange={(e) => setForm({ ...form, url: e.target.value })}
          className="flex-1 rounded-lg border border-gray-300 px-3 py-2 text-sm focus:outline-hidden"
          required
        />
        <button type="submit" className="rounded-lg bg-purple-600 px-4 py-2 text-sm font-semibold text-white hover:b
          Salvar
        </button>
      </form>

      <ul className="space-y-2">
        {favoritos.map((fav) => (
          <li key={fav._id} className="flex items-center justify-between rounded-lg border border-gray-200 p-3">
            <span className="font-medium text-gray-700">{fav.titulo}</span>
            <a href={fav.url} target="_blank" rel="noreferrer" className="text-sm text-blue-600 hover:underline">
              Acessar
            </a>
          </li>
        ))}
      </ul>
    </div>
  );
}
```
